# Biomedical Image-Analysis Pipeline for Fluorescence Microscopy Nuclei

*Tamanna Pathak*

## 1. Overview and Methods

The dataset are fluorescence microscopy images of cell nuclei that are split into 80 training images, 20 validation images, and 12 test images, plus a small extra set of corrupted test images for the robustness bit. I have run three different methods on this data and compared them. The first method is a vision-language model looking at the image directly; the second one is classical Otsu thresholding; and the last one is a trained U-Net doing the same numbers and LLM-after approach but with a proper segmentation model instead of a fixed threshold. The reason for combining all three is basically to see whether letting an LLM look at an image directly is more trustworthy compared to giving it only numbers and then to see how much a trained model improves on that numbers-based approach. Every image ends up going through the same four steps. It gets segmented into a mask; then that mask turns into a table of measurements like area and shape. Those measurements get written up as a short structured JSON record, and then a one- or two-sentence plain-language summary gets generated from that.

## 2. Task 1: Data Preparation and Multimodal LLM Description

### 2.1 EDA

Looking at the histogram, we can see that there's a tall spike right at the very low end, then bars step down, which is followed by a long stretch of low, scattered bars way out at 15-90 with a couple of small bumps around 55-90. That's exactly how the two population patterns really are. The huge spike at the bottom is what background pixels represent, and the sparse tail out to the right is nucleus pixels. They're clearly separated with a gap between them rather than blending into one smooth curve. This is the condition that Otsu thresholding actually needs to work well, since Otsu finds a single cut point that best separates two intensity populations. The shape the histogram has, which is the spike and separated tail, names the two populations it corresponds to, background vs nuclei, and then connects that to why it justifies Otsu specifically. That is because Otsu needs a bimodal-ish distribution with separable peaks; a dominant low peak plus a distinct tail counts.

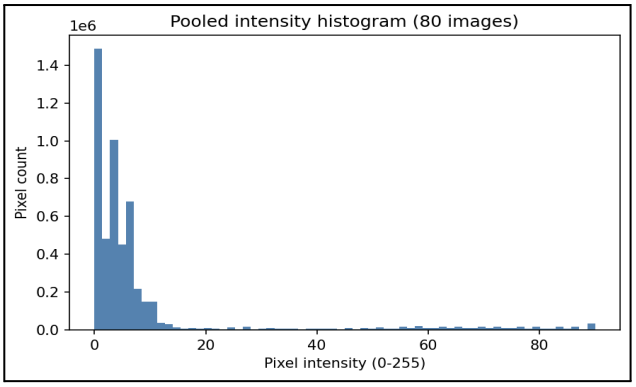


*Figure 1: Pooled pixel intensity histogram across N images*

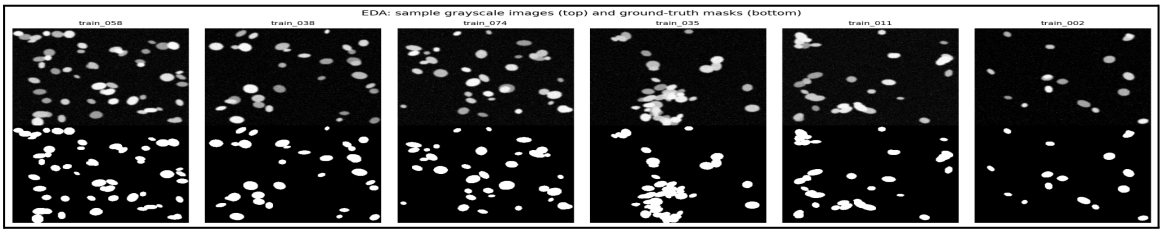


*Figure 2: Random sample of N training images with their corresponding ground-truth segmentation masks*

For the sample grid, it is worth looking at whether the masks, bottom row, line up with the brighter blobs in the raw images, top row, above them since that's the sanity check that the ground truth actually matches the nuclei. Figure 2 represents two things at once. First, the masks correspond pixel-for-pixel to real structures in the raw image; the white blobs in the mask should sit exactly where the brighter nuclei are in the greyscale image. Second, if you look across the columns, some natural variety in the dataset itself is represented by different numbers of nuclei and different sizes/shapes/clustering from one sample to the next.

### 2.2 Optimized prompt

```
You are assisting a researcher by describing a microscopy image. You are NOT a diagnostic tool and must not provide a clinical diagnosis. Look only at what is visible in the image. Respond with ONLY a single valid JSON object (no markdown fences, no extra commentary) with exactly these
```

fields; {"modality": string, // e.g. "fluorescence microscopy", "unknown" "tissue_type": string, // e.g. "cell nuclei", "unknown" "notable_features": [string], // up to 4 short noun phrases you can see "image_quality": string // one of: "good", "acceptable", "poor", "uncertain"

Rules:

- If you are not confident about a field, use the string "uncertain" (or a empty list for notable_features) rather than guessing.

- Do not invent details you cannot see in the image.

- Do not provide any diagnosis, disease name, or clinical recommendation.

- Output must be valid JSON and nothing else

### 2.3 Naive vs. engineered prompt

=== NAIVE PROMPT OUTPUT ===

This image appears to be a microscopic view of something, possibly cells or bacteria, given the circular shapes that resemble cells. The blue colouration suggests that the cells or particles may be stained with a blue dye, which is often used in microscopy to visualise nucleic acids or specific proteins. The circular shapes are typically nuclei, and they are often used to count cells in microscopy experiments.

=== ENGINEERED PROMPT OUTPUT ===

| modality | tissue_type | notable_features | image_quality |
|---|---|---|---|
| fluorescence microscopy | cell nuclei | [blue dots, cells, nuclei] | good |

## 3. Task 2 Classical Features and Numbers—First LLM Interpretation

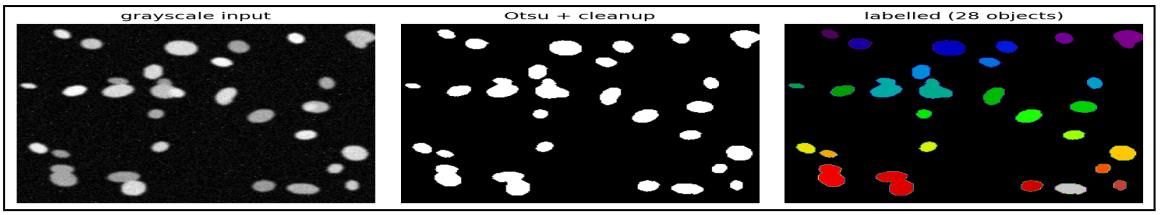


*Figure 3: Classical Otsu pipeline on a representative image (train_003)*

Applying Otsu thresholding, morphological cleanup, and connected-component labelling to the representative image (train_003), 28 objects were detected, with a mean area of 238.6 px and a mean eccentricity of 0.65. The full table is present in the CSV formed, which is task2_feature_table_example.csv. The sample image, train_003, contains a moderate number of nuclei candidates, with an average area of approximately 239 pixels and a median area of around 200 pixels. The nuclei have a high degree of solidity, with a mean value of about 0.95, suggesting they are tightly packed and do not touch each other. The mean intensity of the nuclei is around 64, which is relatively low, indicating they are not very bright. The total area covered by the nuclei in the image is about 10.2% of the entire image, which is considered a moderate foreground area fraction. The result “good" lines up with the high solidity and moderate eccentricity, and "normal" density is consistent with 28 objects covering 10.2% of the image. None of it looks disconnected from the actual numbers, so the results are actually meaningful and give us information.

### Task 1 vs Task 2 comparison

In both tasks, the quality is "good"; that is a genuinely important point since Task 1 is judging visually and Task 2 is judging from measurement, and they land on the same answer. On the other hand, Task 1's notable features, like "blue dots, cells, and nuclei," are vague and don't state a count, density, or shape judgement, so there's nothing concrete there to confirm or contradict the 28 object count, "normal" density, or "regular" shape. It's not wrong; it's just not specific enough to check against the numbers.

To answer which one is better, I would go for Task 2. That's because in task 1, the same image and same model, when run in three separate calls, flip the image quality from "poor" to "good" to "good" with nothing about the input changing. It means that the model directly contradicts itself on the same question. Task 2's numbers don't have that problem because they're computed from pixel measurements, not generated. However, I am not saying the VLM is useless. It's still worth having for a genuinely qualitative description a person might want in plain language, like actually usable output even when you have no computed features yet, or as a rough sanity check.

## 4. Task 3—U-Net Segmentation

In the UNet prediction picture, we have validation images with per-image Dice coefficients of 0.987, 0.987, 0.993, and 0.996. Notably, val_013 is the most crowded image, as it has tightly packed nuclei and still scores 0.993. This is the evidence that the model holds up on the harder, denser case, not just the easy sparse ones. Visually, the predicted masks track the ground truth ellipse shapes and positions closely across all four images. If you look closely at val_013, the predicted blobs are slightly more angular or less smooth at the edges than the ground truth ovals, which is a genuine, minor limitation.

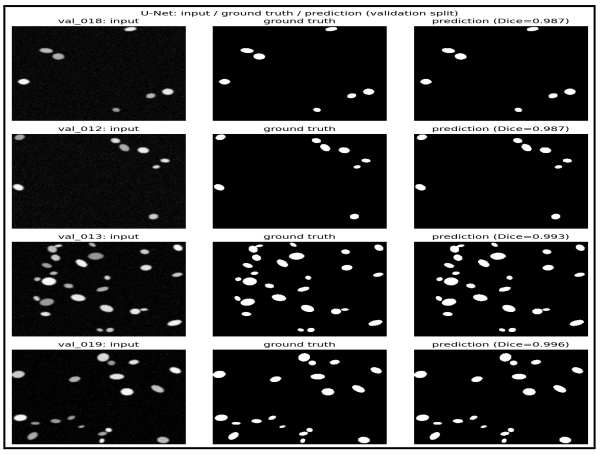


*Figure 4: U-Net predictions on four validation images*

**Talking about the loss curve**, train loss starts at ≈0.7, and val loss starts higher at ≈0.8. Both decrease smoothly and steadily across all 15 epochs, converging by epoch ≈ 3-4 to nearly the same value, and from there, val loss actually tracks slightly below train loss all the way to the end. There is no divergence between train and val, which is the key thing to note here, since a widening gap would signal overfitting, and this doesn't show one.

**The Dice/IoU curve has** a sharp initial jump. Dice goes from ≈0.15 to ≈0.9 and IoU from ≈0.07 to ≈0.9 in just the first 2 epochs. Then there's a small dip around epochs 3-4 where Dice goes down to ≈0.9 and IoU to ≈0.8 before it recovers and climbs steadily to a final Dice of ≈0.99 and IoU of ≈0.98-0.99 by epoch 14. A brief, real wobble mid-training is also seen, which tells us how the curve is not perfectly monotonic.

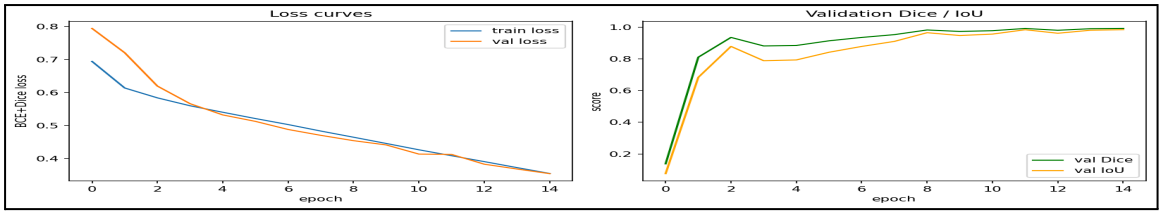


*Figure 5: Training and validation loss (BCE+Dice) alongside validation Dice/IoU over 15 epochs*

## Otsu vs. U-Net

The Otsu picks one global brightness cutoff per image. It can't use shape or local context, so noise pixels above that cutoff get treated the same as real nuclei. The U-Net learns local spatial patterns from training examples, so it can tell a nucleus-shaped blob from a stray speckle even at similar brightness. According to me, that's the core reason it wins. This dataset is also comparatively easy for the method. Synthetic, high-contrast, well-separated nuclei, which is close to Otsu's best-case scenario. Also, worth flagging is the fact that, on messier real microscopy data with lower contrast, touching nuclei, and uneven lighting, Otsu would likely drop off more than U-Net, which is probably widening the gap, but neither method's exact numbers here should be assumed to hold on harder, real data.

## Where the model makes mistakes

Both images from Figure 4 have one thing in common. We see a cluster of 2-3 nuclei sitting close together (val_018 has a touching pair near the top; val_012 has a tighter group of 3 different-sized blobs in the top-right), while the rest of each image is isolated, well-spaced dots. At the pixel level, the prediction masks do locate all the blobs in the right places, and nothing looks fully missed. The value of dice is 0.986-0.987, versus 0.996-0.993 for the other two, with no obviously missing object points toward boundary-level imprecision on exactly those touching clusters rather than a clean whole-object miss. When two nuclei sit close together, the model has less margin to draw a precise boundary between "this pixel belongs to nucleus A" vs "nucleus B" vs

"background", so small edge errors there cost more dice than the same-size error would on an isolated blob. The error concentrates on the boundary between touching nuclei.

## 5. Task 4 - Hybrid Pipeline on the Test Set

The hybrid pipeline was run across all 12 test images, producing a schema-validated JSON record and narrative for each (aggregated in hybrid_pipeline_results.csv); every row required a live, successful Ollama response, since the pipeline raises an error and writes nothing if a call fails.

{"image_id": "test_011", "n_objects": 20, "mean_area": 187.05, "density_class": "sparse", "quality_flag": "good", "narrative": "The segmentation model detected 20 nuclei in the test_011 image, with an average area of 187.1 pixels and a mean eccentricity of 0.63, indicating they are mostly circular. The nuclei have a high degree of solidity, with an average intensity of 65.1 and a total foreground area fraction of 0.057."}

## 6. Critical Discussion

The numbers-first approach, which is Task 2, is more trustworthy than the VLM description, which is Task 1, because every claim it makes can be traced back to a computed value. For instance, on train_003, Task 2's output is grounded directly in the feature table with 28 objects and a mean solidity of 0.95. On the other hand, the VLM output has no equivalent basis. It is useful as a plain-language read. Not as reliable evidence. The JSON schema and numbers prompts used in Tasks 2 and 4 limit what the model can invent; by construction, object count, mean area, and density are computed in code rather than asked of the LLM. This prevents the model from fabricating figures.

The same trustworthiness gap shows up in segmentation. The U-Net performs overall with a mean Dice/IoU of 0.992/0.984 on the validation set and 0.991/0.983 on the test set. For example, val_001 alone scores 0.9895. However, on test_000 under low-contrast corruption, the U-Net's mask collapses entirely to the background with a Dice score of 0.046. This is because it has learned to be tuned to the training distribution that breaks down under distribution shift. On the other hand, Otsu still finds all 8 objects in the same low-contrast image with a Dice score of 0.984. This is because it applies a threshold rule rather than a learned prior, making it the more defensible method in isolation, fully inspectable even when it performs worse on average.

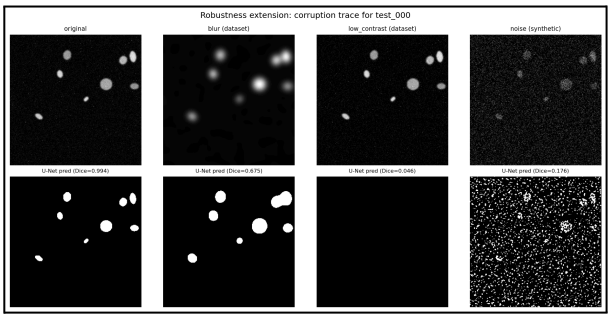


*Figure 6: U-Net predictions on test_000 under three corruption types compared to the original*

The U-Nets numbers can support a research tool with review but not an autonomous one. The single highest-impact fix would be an out-of-distribution check on the U-Nets input. The system can flag when it is operating outside conditions it was trained for. The test_000 failure shows the model has no way of signalling that its own output should be distrusted even though the failure is severe, with a DICE score near zero, not subtle. Now neither Task 2 nor the U-Net has any built-in mechanism to flag when the U-Nets output or Task 2's output should be distrusted.

## 7. References

Ronneberger, O., Fischer, P. and Brox, T., 2015. U-Net: Convolutional Networks for Biomedical Image Segmentation. *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, LNCS 9351, pp.234–241.

Otsu, N., 1979. A threshold selection method from gray-level histograms. *IEEE Transactions on Systems, Man, and Cybernetics*, 9(1), pp.62–66.

Ollama, 2024. *Ollama: Get up and running with large language models locally* [software]. Available at: https://github.com/ollama/ollama [Accessed 19 August 2026].

van der Walt, S., Schönberger, J.L., Nunez-Iglesias, J., Boulogne, F., Warner, J.D., Yager, N., Gouillart, E., Yu, T. and the scikit-image contributors, 2014. scikit-image: image processing in Python. *PeerJ*, 2, p.e453.